

Санкт-Петербургский Национальный Исследовательский Университет ИТМО

Факультет прикладной информатики

Лабораторная работа №8
Разработка задач и анализ алгоритмов

Выполнил:
Сакулин И. М.
Проверил
Мусаев А. А.

Санкт-Петербург,
2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Алгоритмы сортировки	4
2. Алгоритмы поиска	7
3. Хэширование	7
4. Жадные алгоритмы	10
5. Динамическое программирование	13
6. Сложность алгоритмов	16
ЗАКЛЮЧЕНИЕ	22

ВВЕДЕНИЕ

Цель: научиться формулировать нетривиальные задачи по ключевым темам алгоритмов, подбирать наилучшие методы их решения и обосновывать свой выбор.

Требования к заданию: составить по 3 **оригинальных** задания с **интересной формулировкой**, соответствующих **среднему или высокому уровню сложности**, на каждую из 6-ти тем: алгоритмы сортировки, алгоритмы поиска, хэширование, жадные алгоритмы, динамическое программирование, сложность алгоритмов.

Для каждой задачи необходимо:

1. **Описать условие** (1–2 абзаца).
2. **Предложить решение** (псевдокод или описание алгоритма).
3. **Обосновать выбор алгоритма** (обоснование эффективности и сложность).

1. Алгоритмы сортировки

Задача 1.

Формулировка задачи:

В клубе "GEEKMO" инвентаризатор Артём постоянно следит за коллекцией карточек покемонов, хранящейся в виде отсортированного массива. Он хранит коллекцию карточек как набор отсортированных блоков (рунов). Каждый новый пакет карточек разбивается на блоки, которые затем добавляются в коллекцию. Если два соседних блока нарушают условие $X \leq Y$ (где X — размер предыдущего блока, Y — следующего), они сливаются в один и сортируются. После каждого пакета выведите:

1. Количество блоков.
2. Максимальный размер блока.

Формат ввода:

- Первая строка: исходный отсортированный массив через пробел.
- Вторая строка: число N — количество пакетиков.
- Следующие N строк: отсортированные массивы чисел через пробел (пакетики).

Формат вывода:

После добавления каждого пакетика выведите два числа: количество рун и максимальный размер руна через пробел.

Алгоритм решения:

1. Хранение блоков: основная коллекция — список отсортированных блоков.
2. Добавление пакета: каждый пакет разбивается на блоки фиксированного размера $\text{minrun}=2$, блоки пакета добавляются в коллекцию.
3. Слияние блоков: пока в коллекции есть два соседних блока, где предыдущий размер $\leq$ следующий размер не выполняется, сливать их в один и сортировать.

Сложность в худшем случае — $O(N \log N)$ из-за сортировки, но количество операций слияния зависит от входных данных.

Задача 2.

Формулировка задачи

Винни-Пух получил от ослика Иа-Иа мешок с волшебными цветами, каждый из которых имеет свой числовой код. Он хочет посадить их вокруг улья так, чтобы сначала шли цветы, которых больше всего. Если два вида цветов встречаются одинаково часто, первым должен идти тот, у которого код больше. Помогите Винни расставить цветы в правильном порядке!

Формат ввода:

На вход подаётся массив целых чисел `flowers` — коды цветов.

Формат вывода:

Выведите массив, отсортированный по убыванию частоты встречаемости цветов. При одинаковой частоте числа идут в порядке убывания.

Алгоритм решения: Counter Sort

```
def sort_flowers(flowers):
    # Подсчёт частот
    frequency = {}
    for num in flowers:
        if num in frequency:
            frequency[num] += 1
        else:
            frequency[num] = 1

    # Сортируем ключи: сначала по убыванию частоты, затем по убыванию значения
    sorted_keys = sorted(frequency.keys(), key=lambda x: (-frequency[x], -x))

    # Собираем результат
    result = []
    for num in sorted_keys:
        result.extend([num] * frequency[num])

    return result

# Чтение ввода и вывод
flowers = list(map(int, input().split()))
sorted_flowers = sort_flowers(flowers)
print(' '.join(map(str, sorted_flowers)))
```

Сложность по времени: $O(n + m \log m)$, где n — длина массива, m — количество уникальных чисел. **Сложность по памяти:** $O(n)$ (для хранения результата и словаря).

Задача 3.

Формулировка задачи

Перед вами множество кучек с конфетами, но злой волшебник подмешал яд в те кучи, где количество конфет делится на его любимое число N . Чтобы спастись, вы должны собрать до M безопасных кучи, начиная с самой большой. Определите порядок сбора.

Формат ввода:

- Первая строка: список целых чисел через пробел — количества конфет в кучах.
- Вторая строка: целое число N (магическое число волшебника) и M через пробел.

Формат вывода:

- Список целых чисел, представляющий порядок сбора конфет от наибольшей к наименьшей среди безопасных куч. Если безопасных куч нет, вывести пустой список.

Алгоритм решения: Heap Sort

```
import heapq

def safe_candy_order(arr, n, m):
    safe = [x for x in arr if x % n != 0]
    if not safe:
        return []
    max_heap = [-x for x in safe]
    heapq.heapify(max_heap)
    result = []
    i = 0
    while max_heap and i <= m:
        result.append(-heapq.heappop(max_heap))
        i += 1
    return result

# Чтение ввода
arr = list(map(int, input().split()))
n, m = map(int, input().split())

# Получение результата и вывод
output = safe_candy_order(arr, n, m)
print(' '.join(map(str, output)) if output else '')
```

1. **Фильтрация:** затрачивает $O(n)$ времени, где n — длина исходного массива.
 2. **Извлечение элементов:** каждая операция `heappop` выполняется за $O(\log k)$, где k — количество безопасных элементов $k \leq M$. Общая сложность — $O(k \log k)$.
- Итоговая сложность:** $O(n + k \log k)$.

2. Алгоритмы поиска

Задача 4

Формулировка задачи

Вы - биолог, исследующий поле одуванчиков. Поле представлено матрицей, где каждая строка соответствует ряду цветов, а числа в строке обозначают дни, когда одуванчики в этом ряду начинают цвести. Ряды отсортированы по времени цветения слева направо. Ваша задача — найти ряд, в котором наибольшее количество одуванчиков зацвели последовательно (например, дни 5, 6, 7, 8). Если таких рядов несколько, выберите самый верхний.

Формат ввода:

- Матрица `matrix` размером $m \times n$, где $1 \leq m, n \leq 10^3$.
- Каждая строка матрицы отсортирована по возрастанию.
- Элементы матрицы — целые числа в диапазоне $[-10^9, 10^9]$.

Формат вывода:

- Индекс строки (считая от 0) с максимальной длиной последовательных чисел. Если несколько строк имеют одинаковую максимальную длину, верните наименьший индекс.

Алгоритм решения:

1. Для каждой строки матрицы вычисляем длину наибольшей последовательности идущих подряд чисел.
2. Поскольку строки отсортированы, последовательные числа должны идти друг за другом. Проходим по элементам строки, сравнивая текущий элемент с предыдущим. Если разница равна 1, увеличиваем счётчик текущей последовательности, иначе сбрасываем его.
3. Сохраняем максимальную длину последовательности для каждой строки. Выбираем строку с наибольшим значением (при равенстве — с меньшим индексом).

```
def find_max_consecutive_row(matrix):
    max_length = -1
    result_row = 0

    for i, row in enumerate(matrix):
        current_len = 1
        row_max = 1
        for j in range(1, len(row)):
            if row[j] == row[j-1] + 1:
                current_len += 1
                row_max = max(row_max, current_len)
            else:
                current_len = 1
        if row_max > max_length:
            max_length = row_max
            result_row = i

    return result_row
```

- **Временная сложность:** $O(m * n)$, где m — количество строк, n — количество элементов в строке. Каждый элемент обрабатывается один раз.
- **Пространственная сложность:** $O(1)$, так как используются только переменные для хранения текущих значений.

Задача 5

Формулировка задачи

Программист Иван коллекционирует пакетики чая с разными вкусами, каждый из которых представлен уникальным целым числом. У Ивана есть начальное количество пакетиков для каждого вкуса. Он хочет быстро обрабатывать два типа операций:

1. **Обновить** количество пакетиков для определённого вкуса.
2. **Посчитать** общее количество пакетиков для всех вкусов в диапазоне $[L, R]$ (включительно).

Помогите Ивану эффективно отвечать на запросы, используя дерево отрезков.

Формат ввода:

- Первая строка содержит число N — количество начальных вкусов.
- Следующие N строк содержат пары чисел $taste_i$ $count_i$ — вкус и количество пакетиков.
- Затем идёт число M — количество запросов.
- Следующие M строк содержат запросы:
 - UPDATE t c — увеличить количество пакетиков вкуса t до c .
 - QUERY L R — вывести сумму пакетиков для всех вкусов между L и R .

Формат вывода:

Для каждого запроса QUERY выведите результат.

```
1.
Собрать все вкусы из начальных данных и запросов UPDATE
Отсортировать вкусы в порядке возрастания
Создать словарь taste_to_idx: вкус → индекс в отсортированном списке

2.
Создать массив data размером с количество уникальных вкусов
Для каждого начального вкуса t:
    data[taste_to_idx[t]] = count_t
Построить дерево отрезков на основе data

3. Обработка запросов
Для каждого запроса:
    Если тип запроса UPDATE:
        t = вкус из запроса
        c = новое количество
        idx = taste_to_idx[t]
        Обновить дерево: tree.update(idx, c)
    Иначе если тип запроса QUERY:
        L, R = границы диапазона
        left = бинарный поиск первого вкуса ≥ L
        right = бинарный поиск последнего вкуса ≤ R
        Если left > right: вывести 0
        Иначе: вывести tree.query_sum(left, right)
```

Сложность

- **Дерево отрезков** обеспечивает обновление и запрос суммы за $O(\log N)$.
- **Бинарный поиск** для определения границ диапазона выполняется за $O(\log N)$.
- Общая сложность: $O((N + M) \log N)$, что эффективно для больших данных.

Задача 6

Формулировка задачи

Вы искатель приключений, запертый в древнем лабиринте, где каждый уровень (ряд) отсортирован по возрастанию. Чтобы открыть выход внизу справа, нужно собрать все ключи, спрятанные в катакомбе. Ключи пронумерованы от 1 до K и расположены в строгом порядке: ключ $i+1$ находится в клетке с большим значением, чем ключ i . Стены (-1) блокируют путь. Найдите минимальное количество шагов для сбора всех ключей и выхода.

Формат ввода:

Двумерный массив `grid`, где:

- 0: Пустая клетка.
- -1: Стена.
- Положительное число: Ключ (уникальный, от 1 до K).

Формат вывода:

Целое число — минимальное количество шагов. Если невозможно, вернуть -1.

Алгоритм решения

1. Найти все ключи и отсортировать их.
2. Инициализировать поиск BFS с начальной позицией (0,0), ключей 0.
3. Для каждого состояния:
 - a. Если достигнут выход и все ключи собраны — вернуть шаги.
 - b. Исследовать соседние клетки.
 - c. При сборе ключа обновить состояние.
4. Если очередь пуста — вернуть -1.

Сложность

- **BFS** гарантирует нахождение кратчайшего пути.
- Состояние учитывает позицию и количество собранных ключей, что ограничивает пространство поиска до $M \cdot N \cdot K$

3. Хэширование

Задача 7

Формулировка задачи

Вы — хранитель древнего свитка, содержащего магические руны. Каждое заклинание кодируется уникальной последовательностью из L символов. Однако сила заклинания пробуждается только если оно встречается в тексте нечётное количество раз (1, 3, 5...). Определите, какие заклинания из списка D активированы в тексте S , учитывая, что руны могут накладываться друг на друга (например, в "aaaa" при $L=2$ подстроки "aa" встречаются 3 раза).

Формат ввода:

- Строка S (текст свитка).
- Список строк D (заклинания, все длины L).
- Целое число L (длина рун).

Формат вывода:

Отсортированный список заклинаний из D , которые встречаются в S нечётное число раз. Если таких нет — вернуть пустой список.

Алгоритм решения:

1. Подсчёт вхождений: с помощью скользящего хеша найти все подстроки длины L в S и подсчитать их частоту в хеш-таблице.
2. Для каждого заклинания из D проверить его наличие в таблице и определить, нечётный ли у него счётчик.
3. Использовать хэширование для быстрого вычисления хешей подстрок.
4. Во избежание коллизий хешей сравнивать подстроки напрямую.

Анализ алгоритма:

- Подсчёт подстрок: $O(N)$, где N — длина текста (скользящий хеш).
- Проверка заклинаний: $O(M * L)$, где M — количество заклинаний (прямое сравнение строк при коллизиях).

Задача 8

Формулировка задачи:

Вадим увлекается созданием коллажей из букв, вырезанных из журналов. Сейчас он хочет составить секретное письмо, используя свою коллекцию. Помогите ему определить, сколько и каких букв не хватает. Если букв хватает, выведите соответствующее сообщение.

Формат ввода:

- Первая строка: символы, которые есть у Вадима (available).

- Вторая строка: символы, необходимые для письма (target).

Формат вывода:

- Выведите буквы и их недостающее количество в лексикографическом порядке (например, А 3).
- Если всех букв хватает, выведите: "Все буквы присутствуют".

Алгоритм работы: подсчёт частот символов, сравнение частот, вывод результата.

```
available = input().strip()
target = input().strip()

target_counts = {}
for c in target:
    target_counts.setdefault(c, []) += 1

available_counts = {}
for c in available:
    available_counts.setdefault(c, []) += 1

missing = []
for char in sorted(target_counts):
    needed = target_counts[char]
    have = available_counts.get(char, 0)
    if have < needed:
        missing.append(f"{char} {needed - have}")

if not missing:
    print("Все буквы присутствуют")
else:
    print("\n".join(missing))
```

Сложность времени: $O(n + m + k \log k)$, где n и m — длины строк, k — количество уникальных символов в target.

Задача 9

Формулировка задачи

Вы защищаете магические артефакты от темных сил. Каждый артефакт имеет силу, выраженную целым числом. Артефакты с силой, кратной p , нестабильны: если в группе артефактов таких больше k , они могут разрушить храм. Ваша задача — найти все **уникальные** комбинации подряд идущих артефактов, где нестабильных элементов не больше k . Две комбинации считаются разными, если они отличаются длиной или хотя бы одним элементом.

Формат ввода:

- `nums` — целочисленный массив ($1 \leq \text{len}(\text{nums}) \leq 200$).
- k — целое число ($0 \leq k \leq \text{len}(\text{nums})$).
- p — целое число ($1 \leq p \leq 200$).

Формат вывода:

- Целое число — количество уникальных подмассивов.

Алгоритм решения

Для массива `nums` и целых чисел `k`, `p` определите количество уникальных подмассивов, в которых не более `k` элементов делятся на `p`.

```
def countDistinct(nums, k, p):
    n = len(nums)
    div = [1 if num % p == 0 else 0 for num in nums]
    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i+1] = prefix[i] + div[i]

    unique = set()

    for l in range(n):
        for r in range(l, n):
            total = prefix[r+1] - prefix[l]
            if total > k:
                break
            subarray = tuple(nums[l : r+1])
            unique.add(subarray)

    return len(unique)
```

Сложность

Время: $O(n^2)$ — в худшем случае перебираются все подмассивы. Память: $O(n^2)$ — в худшем случае все подмассивы уникальны.

4. Жадные алгоритмы

Задача 10

Вы – главный хирург в перегруженном госпитале. Каждый пациент требует операции, но задержки приводят к штрафам, которые больница хочет минимизировать. У пациента i операция занимает t_i минут, должен быть прооперирован до времени d_i , иначе за каждую минуту задержки начисляется штраф s_i . Врач может оперировать только одного пациента за раз. Определите порядок операций, который минимизирует суммарный штраф.

Формат ввода:

- Первая строка: целое число n (количество пациентов).
- Следующие n строк: три целых числа t_i , d_i , s_i (время операции, дедлайн, штраф за минуту задержки).

Формат вывода:

Минимальный суммарный штраф.

Алгоритм решения:

```
Сортировать пациентов по убыванию  $s_i / t_i$ 
current_time = 0
штраф = 0
для каждого пациента в отсортированном списке:
    finish_time = current_time + t_i
    задержка = max(0, finish_time - d_i)
    штраф += задержка * s_i
    current_time = finish_time
результат: штраф
```

Жадный алгоритм сортирует пациентов по убыванию отношения штрафа к времени операции (s_i/t_i). Этот выбор минимизирует "плотность" накопления штрафа за единицу времени. Доказательство оптимальности: если в оптимальном решении существует пара пациентов, нарушающих порядок сортировки, их замена уменьшит штраф.

Задача 11

Семён составил чёткий план питания на n дней, где каждый день он ест либо суп (обозначается 'S'), либо макароны (обозначается 'P'). У него есть m чистых глубоких тарелок (для супа) и k чистых плоских (для макарон). После использования тарелка становится грязной. В конце любого дня Семён может помыть все грязные тарелки одного или обоих типов, что вернёт их в чистые. Каждый день мытья (независимо от количества вымытых

тарелок) считается как одно действие. Определите минимальное количество дней, в которые нужно помыть тарелки, чтобы Семён смог следовать своему плану.

Формат ввода:

Первая строка: три целых числа m , k , n (количество глубоких, плоских тарелок и дней).

Вторая строка: строка длины n , состоящая из символов 'S' и 'P' (план питания).

Формат вывода:

Одно целое число — минимальное количество дней с помывкой.

Решение:

```
m, k, n = map(int, input().split())
plan = input().strip()

clean_deep = m
clean_flat = k
dirty_deep = 0
dirty_flat = 0
wash_days = 0

for i in range(n):
    dish = plan[i]

    # Проверяем, достаточно ли чистых тарелок
    if dish == 'S' and clean_deep == 0:
        # Нужно помыть все глубокие
        clean_deep += dirty_deep
        dirty_deep = 0
        wash_days += 1
    elif dish == 'P' and clean_flat == 0:
        # Нужно помыть все плоские
        clean_flat += dirty_flat
        dirty_flat = 0
        wash_days += 1

    # Используем тарелку
    if dish == 'S':
        clean_deep -= 1
        dirty_deep += 1
    else:
        clean_flat -= 1
        dirty_flat += 1

print(wash_days)
```

Доказательство оптимальности: мы моем тарелки только тогда, когда это абсолютно необходимо, что минимизирует количество операций.

Задача 12

На день рождения подружки вы решили сделать шоколадный дом, вы начали с пола, который представляет собой сетку $M \times N$, где некоторые клетки разрушены (обозначены '#'), а остальные целы ('.'). Необходимо покрыть максимальное количество целых клеток плитками шоколада 2×1 , ведь подружка очень любит шоколад. Но есть ограничения:

1. Плитки можно располагать только **горизонтально**.
2. Каждая плитка должна покрывать ровно одну "светлую" и одну "тёмную" клетку (сетка раскрашена в шахматном порядке).

Найдите максимальное количество плиток, которое можно уложить, соблюдая правила.

Формат ввода:

Первая строка: два целых числа M и N (размер сетки).

Следующие M строк: строка из N символов ('.' — целая клетка, '#' — разрушенная).

Формат вывода:

Одно число — максимальное количество плиток.

Алгоритм решения

1. Раскрасить сетку в шахматном порядке.
2. Для каждой строки подсчитать количество пар соседних клеток (i, j) и $(i, j+1)$, где одна светлая, другая тёмная, и обе целые.
3. Суммировать все такие пары по строкам.

Корректность.

Каждая пара учитывается ровно один раз, плитки не перекрываются.

Оптимальность.

В каждой строке жадно берём максимальное возможное число пар. Горизонтальное расположение гарантирует, что выбор в одной строке не влияет на другие.

Сложность: $O(M * N)$ — каждый элемент обрабатывается один раз.

5. Динамическое программирование

Задача 13

Формулировка задачи:

Лягуш отправляется на поиски самого вкусного болота. Перед ней сетка кувшинок размером $m \times n$. Каждая кувшинка содержит некоторое количество мух или жуков-навозников (целое число, положительное – мухи, отрицательное – жуки). Лягуш начинает в левом верхнем углу $(0, 0)$ и хочет добраться в правый нижний угол $(m-1, n-1)$, прыгая только вправо или вниз. Однако, если Лягуш съест жуков-навозников с кувшинки с отрицательным значением, у неё пропадёт аппетит, и на следующей кувшинке она не сможет есть букашек. Какую максимальную сумму мух может собрать Лягуш?

Формат ввода:

В первой строке два целых числа m и n — размеры сетки.

Следующие m строк содержат по n целых чисел, разделённых пробелами — значения на кувшинках.

Формат вывода:

Одно целое число — максимальное количество мух, которое можно собрать. Если путь невозможен, вывести $-\text{inf}$ (например, если начальная или конечная клетка заблокированы).

Алгоритм решения

Используем динамическое программирование с двумя состояниями для каждой клетки:

- $dp[i][j][0]$ — максимальная сумма при достижении (i, j) без активного "штрафа" (можно есть букашек здесь).
- $dp[i][j][1]$ — максимальная сумма при достижении (i, j) с активным штрафом (нельзя есть здесь).

```
массив dp[m][n][2] = {-inf}
dp[0][0][0] = grid[0][0], если grid[0][0] >= 0
dp[0][0][1] = 0, если grid[0][0] < 0

Для каждой клетки (i, j) по порядку:
    Для каждого направления (сверху или слева):
        Если предыдущая клетка (k, l) существует:
            Для обоих состояний (0 и 1):
                Если возможно перейти в (i, j) с учётом правил:
                    Обновить dp[i][j][new_state] максимумом

Ответ — максимум из dp[m-1][n-1][0] и dp[m-1][n-1][1]
```

Временная сложность: $O(mn)$, так как для каждой из mn клеток мы рассматриваем два состояния.

Задача 14

Программист Сергей любит линукс, поэтому во всех своих сообщениях хочет сжать текст, заменяя технические термины на команды `bush`. Каждое слово из словаря можно заменить на один символ. Найдите минимальную длину сжатого текста.

Формат ввода:

Строка S

Словарь D — список подстрок ($1 \leq |D| \leq 100$, длина каждой подстроки от 1 до 100).

Формат вывода:

Минимальная длина сжатого текста.

Алгоритм:

Группируем подстроки по длинам, для этого создаем словарь, где ключ — длина, значение — множество подстрок этой длины.

$dp[i]$ — минимальная длина для первых i символов.

Базовый случай: $dp[0] = 0$.

Для каждого символа проверяем все возможные подстроки из словаря, заканчивающиеся на текущем символе, и обновляем dp .

```
def minimal_encoded_length(S, D):
    len_to_subs = {}
    for sub in D:
        len_to_subs.setdefault(len(sub), set()).add(sub)
    lengths = list(len_to_subs.keys())
    n = len(S)
    dp = [float('inf')] * (n + 1)
    dp[0] = 0

    for i in range(1, n + 1):
        dp[i] = dp[i-1] + 1
        for l in lengths:
            if l > i:
                continue
            if S[i-l:i] in len_to_subs[l]:
                dp[i] = min(dp[i], dp[i-l] + 1)
    return dp[n]
```

Сложность.

Временная: $O(n * k)$, где k — число уникальных длин в D . Эффективно при небольших k . **Пространственная:** $O(n)$ для хранения dp .

Задача 15

Артём мечтает построить самый красивый замок в песочнице. Его грузовичок может перевезти **ровно K камней**, и их суммарный объём не должен превысить **V**. Каждый камень обладает уникальной «красивостью» и занимает место в кузове. Помогите Артёму выбрать **ровно K камней** так, чтобы их общая красивость была максимальной, соблюдая ограничения грузовичка. Если невозможно выбрать ровно K камней, не превысив объём V, выведите 0.

Формат ввода:

- Первая строка содержит три целых числа: **N** (количество камней), **K** (требуемое количество камней), **V** (максимальный объём).
- Следующие **N** строк описывают камни: каждое содержит два числа **c_i** (красивость) и **v_i** (объём).

Формат вывода:

- Максимальная суммарная красивость ровно K камней, если их объём $\leq V$. Иначе 0.

Алгоритм решения:

Динамическое программирование с состоянием $dp[k][v]$ — максимальная красивость при выборе **ровно k** камней с суммарным объёмом **v**.

1. $dp[0][0] = 0$ (0 камней, объём 0), остальные $dp[k][v] = -\infty$.
2. Для каждого камня перебираем **k** от K вниз до 1, и **v** от V вниз до v_i .
3. Ответ — максимум среди $dp[K][v]$ для всех $v \leq V$. Если максимум отрицательный (недостижимо), выводим 0.

Сложность

Сложность по времени: $O(N \times K \times V)$. Сложность по памяти: $O(K \times V)$.

6. Сложность алгоритмов

Задача 16

Вадим получил мешок конфет, где каждая конфета имеет определенный вкус (представленный целым числом). Он хочет оставить себе все конфеты доминирующего вкуса, который встречается более чем в четверти случаев, а остальные отдать брату Ивану. Помогите Вадиму определить такой вкус, если он существует, используя алгоритм с линейным временем работы и константной памятью.

Формат ввода:

Строка с целыми числами через пробел — идентификаторы кандидатов.

Формат вывода:

- Список целых чисел, отсортированный по возрастанию, — конфеты, встречающиеся более $n/4$ раз.
- Если таких нет, выведите -1.

Алгоритм

Используется модификация алгоритма Бойера-Мура для отслеживания **трех** кандидатов, так как более трех элементов не могут одновременно встречаться $> n/4$ раз.

```
def find_council_members(arr):
    count1, count2, count3 = 0, 0, 0
    cand1 = cand2 = cand3 = None

    for num in arr:
        if num == cand1:
            count1 += 1
        elif num == cand2:
            count2 += 1
        elif num == cand3:
            count3 += 1
        elif count1 == 0:
            cand1, count1 = num, 1
        elif count2 == 0:
            cand2, count2 = num, 1
        elif count3 == 0:
            cand3, count3 = num, 1
        else:
            count1 -= 1
            count2 -= 1
            count3 -= 1

    # Проверяем частоты конфет
    threshold = len(arr) // 4
    result = []
    candidates = {cand1, cand2, cand3}
    for cand in candidates:
        if cand is not None and arr.count(cand) > threshold:
            result.append(cand)

    result = sorted(result) if result else [-1]
    return result if result[0] != -1 else [-1]
```

Время: Два прохода по списку ($O(n)$). **Память:** Три счетчика и три кандидата ($O(1)$).

При использовании хеш-таблицы: $O(n)$ **памяти**.

Задача 17

Аналитики из университета Л обошли магазины радиоэлектроники и собрали данные о возрастных диапазонах клиентов, которые активно совершали покупки. Каждый клиент указал возраст, в котором начал (включительно) и перестал (не включительно) быть активным. Определите, в каком возрасте одновременно было максимальное количество активных клиентов.

Формат ввода:

Список интервалов в формате $[[start1, end1], \dots]$, где $start$ и end — целые числа (возраст в годах).

Формат вывода:

Максимальное количество активных клиентов в одном возрасте.

Алгоритм:

1. Преобразовать каждый интервал $[start, end)$ в два события: $(start, +1)$ (начало активности) и $(end, -1)$ (конец активности).
2. Отсортировать события по возрасту. При равенстве возрастов сначала обрабатываются события с -1 , чтобы корректно закрывать интервалы.
3. Пройти по отсортированным событиям, обновляя счетчик активных клиентов. Зафиксировать максимальное значение счетчика.

Сложность: Время: $O(n \log n)$ из-за сортировки. Память: $O(n)$ для хранения событий.
Альтернатива: Перебор всех временных точек — $O(n^2)$.

Задача 18

Где-то за Уральскими горами существует город, в котором все перекрёстки соединены односторонними дорогами. Каждая дорога имеет определённое время проезда. Депутат Н хочет попасть домой как можно быстрее. Патруль ДПС может изменить направление одной дороги. Определите минимально возможное время пути с учётом этой «магической» модификации.

Формат ввода

Первая строка содержит четыре целых числа: n (количество перекрёстков), m (количество дорог), A (стартовый перекрёсток), B (конечный перекрёсток). Следующие m строк содержат по три целых числа u, v, w , обозначающих дорогу из перекрёстка u в перекрёсток v с временем проезда w .

Формат вывода

Выведите одно целое число — минимальное время пути. Если добраться невозможно даже с изменением направления дороги, выведите -1 .

Алгоритм решения.

1. С помощью алгоритма Дейкстры найти кратчайшие расстояния от A до всех городов (dist_A).
2. Обратить все дороги и найти кратчайшие расстояния от B до всех городов в обращённом графе (dist_B).
3. Для каждой дороги $u \rightarrow v$ с весом w рассмотреть путь, использующий перевёрнутую дорогу: $A \rightarrow \dots \rightarrow v \rightarrow u \rightarrow \dots \rightarrow B$.
4. Вычислить длину пути как $\text{dist_A}[v] + w + \text{dist_B}[u]$.
5. Минимальный путь будет наименьшим значением между исходным кратчайшим путём и минимальным значением из перебора рёбер.

Сложность алгоритма Дейкстры: $O(M + N \log N)$ для каждого вызова, где N — число городов, M — число дорог. Перебор рёбер: $O(M)$ операций. В итоге $O(M + N \log N)$.

Наивное решение ($O(m * (n + m))$): для каждого ребра ($u \rightarrow v$) попробовать обратить его на ($v \rightarrow u$) и запустить алгоритм Дейкстры или BFS (если веса единичные).

ЗАКЛЮЧЕНИЕ

Я попробовал формулировать нетривиальные задачи по ключевым темам алгоритмов, подбирать методы их решения и обосновывать свой выбор. На первых задачах ещё был энтузиазм и новые решения, но для последних задач пришлось очень сильно вдохновляться существующими.